```c
/* Switch the current execution context to the f_name.
 * Returns -1 on fail. */
// 현재 프로세스를 새 사용자 프로그램으로 갈아끼우는 함수
int
process_exec (void *f_name) {
    // 커맨드 라인 받기
    // 여기서 원본 메모리가 팔록으로 할당된 버퍼라서 이후에 계속 할당 해제해줘야함
    char *file_name = f_name;
    bool success;

    // file_name이 유효한 포인터 주소인지 먼저 검사한다.
    if (file_name == NULL){
        return -1;
    }

    // 길이가 너무 길지 않은지 확인한다.
    if(strnlen(file_name, PGSIZE) == PGSIZE){
        palloc_free_page (file_name);
        return -1;
    }

    // 공백만 있는 입력인지 확인한다.
    if(strspn(file_name, " ") == strlen(file_name)){
        palloc_free_page (file_name);
        return -1;
```

```c
    }

    // 기능 1: 커맨드라인 파싱 (토큰화)
    // process_exec() 파싱 시작부

    // 쓰기 가능한 복사 버퍼를 할당한다.
    char *cmd_line = palloc_get_page (0);
    if (cmd_line == NULL){
        palloc_free_page (file_name);
        return -1;
    }

    // 복사 버퍼에 복사한다.
    strlcpy(cmd_line, file_name, PGSIZE);

    // parse_command_line_args() 토큰화 루프 진입전 변수할당
    int argc = 0;
    char **argv = palloc_get_page (0);

    // 할당된 배열이 유효한지 확인
    if (argv == NULL){
        palloc_free_page (file_name);
        palloc_free_page (cmd_line);
        return -1;
    }
```

```
643
644     // parse_command_line_args() 토큰화 루프 진입
645     // 공백기준으로 토큰을 추출한다
646     success = parse_command_line_args(cmd_line, &argc, argv);
647     if (!success){
648         palloc_free_page (file_name);
649         palloc_free_page (cmd_line);
650         palloc_free_page (argv);
651         return -1;
652     }
653
654     // finalize_parsed_args() 파싱 결과 반환
655     success = finalize_parsed_args(argc, argv);
656     if (!success){
657         palloc_free_page (file_name);
658         palloc_free_page (cmd_line);
659         palloc_free_page (argv);
660         return -1;
661     }
662
663     /* We cannot use the intr_frame in the thread structure.
664      * This is because when current thread rescheduled,
665      * it stores the execution information to the member. */
666     struct intr_frame _if;
667     _if.ds = _if.es = _if.ss = SEL_UDSEG;
```

```
668     _if.cs = SEL_UCSEG;
669     _if.eflags = FLAG_IF | FLAG_MBS;
670
671     /* We first kill the current context */
672     // 현재 스레드의 running_file 포인터를 확인하고 해제한다.(사용중인 파일에 대한 쓰기 금지 해제)
673     close_running_file(thread_current());
674     // 유저용 데이터를 완전히 비워준다.(pm_14, PML4 (Page Map Level 4))
675
676     /*1. PML4의 직관적 구조 (4단계 계층)
677
678     - 64비트 가상 주소는 64개 비트를 다 쓰지 않고 하위 48비트만 사용하는데, 이를 9비트씩 4등분하여 각 단계의 인덱스로 활용한다.
679
680     1단계: PML4 (Page Map Level 4)
681
682     - 최상위 목차다. 512개의 엔트리를 가지며, 각 엔트리는 다음 단계인 PDPT의 주소를 가리킨다.
683     - CPU의 CR3 레지스터가 이 PML4의 시작 위치를 항상 기억하고 있다.
684
685     2단계: PDPT (Page Directory Pointer Table)
686
687     - PML4의 지시를 받아 다음 단계인 PD의 주소를 가리킨다.
688
689     3단계: PD (Page Directory)
690
691     - 다음 단계인 PT의 주소를 가리킨다.
692
```

```
4단계: PT (Page Table)

- 가장 하위 단계의 지도로, 여기서 드디어 실제 데이터가 저장된 물리 메모리(Frame)의 주소를 알려준다.

2. 왜 굳이 4단계나 나누는가?

이 방식의 핵심은 "필요한 부분만 만든다"는 것이다.
- 메모리 절약: 유저 프로그램이 가상 주소 $2^{48}$바이트 전체를 다 쓰는 경우는 거의 없다. 사용하지 않는 주소 영역은 상위 단계(PML4나 PDPT)의 엔트리를 NULL로 비워두기만 하면 된다. 그 아래 단계의 표들은 아예 생성할 필요가 없으므로 메모리가 획기적으로 절약된다.
- 권한 관리: 각 단계의 엔트리마다 "이 영역은 읽기 전용인가?", "유저가 접근 가능한가?" 같은 권한 비트를 심어둘 수 있다. 하드웨어가 주소를 찾아 내려가다가 권한이 없는 층을 발견하면 즉시 차단(Segmentation Fault 등)한다.*/

process_cleanup ();

// TODO: Argument 분리해서 파일명만 load()로 넘기기
// 기능 2: 사용자 스택 레이아웃 구성 부분 시작 (ABI 계약)

// build_user_stack_args()가 계산한 argv 주소를, 다음 단계인 set_user_entry_registers()에 전달하기 위한 중간 저장소
uintptr_t argv_user_addr = 0;

// load() 기본 매핑
success = load(argv[0], &_if);
if (!success){
    palloc_free_page (file_name);
    palloc_free_page (cmd_line);
```

```
        palloc_free_page (argv);
        return -1;
    }
    else{
        // build_user_stack_args() 인자 배치 루프
        success = build_user_stack_args(&_if, argc, argv, &argv_user_addr);
        if (!success){
            palloc_free_page (file_name);
            palloc_free_page (cmd_line);
            palloc_free_page (argv);
            return -1;
        }
    }

    // 기능 3: 레지스터 설정과 유저 진입 경계
    // set_user_entry_registers() intr_frame 인자 필드
    success = set_user_entry_registers(&_if, argc, argv_user_addr);
    if (!success){
        palloc_free_page (file_name);
        palloc_free_page (cmd_line);
        palloc_free_page (argv);
        return -1;
    }

    // validate_user_entry_frame() 유저 전환 직전 점검
```

```
    success = validate_user_entry_frame(&_if);
    if (!success){
        palloc_free_page (file_name);
        palloc_free_page (cmd_line);
        palloc_free_page (argv);
        return -1;
    }

    // do_iret() 유저 진입
    palloc_free_page (file_name);
    palloc_free_page (cmd_line);
    palloc_free_page (argv);
    do_iret (&_if);
    NOT_REACHED ();
}
```